

Mobile Devices and Applications

Unit 10 – Persistent storage

Agenda

Persistent storage

- Application container
- Shared Preferences files
- Internal and External files
- Database - SQLite

Content Providers

DATA STORAGE



Why Persistency Matters



RETAINS USER DATA
ACROSS SESSIONS



SUPPORTS OFFLINE
FUNCTIONALITY



IMPROVES APP
RELIABILITY



ENABLES COMPLEX
DATA MODELING

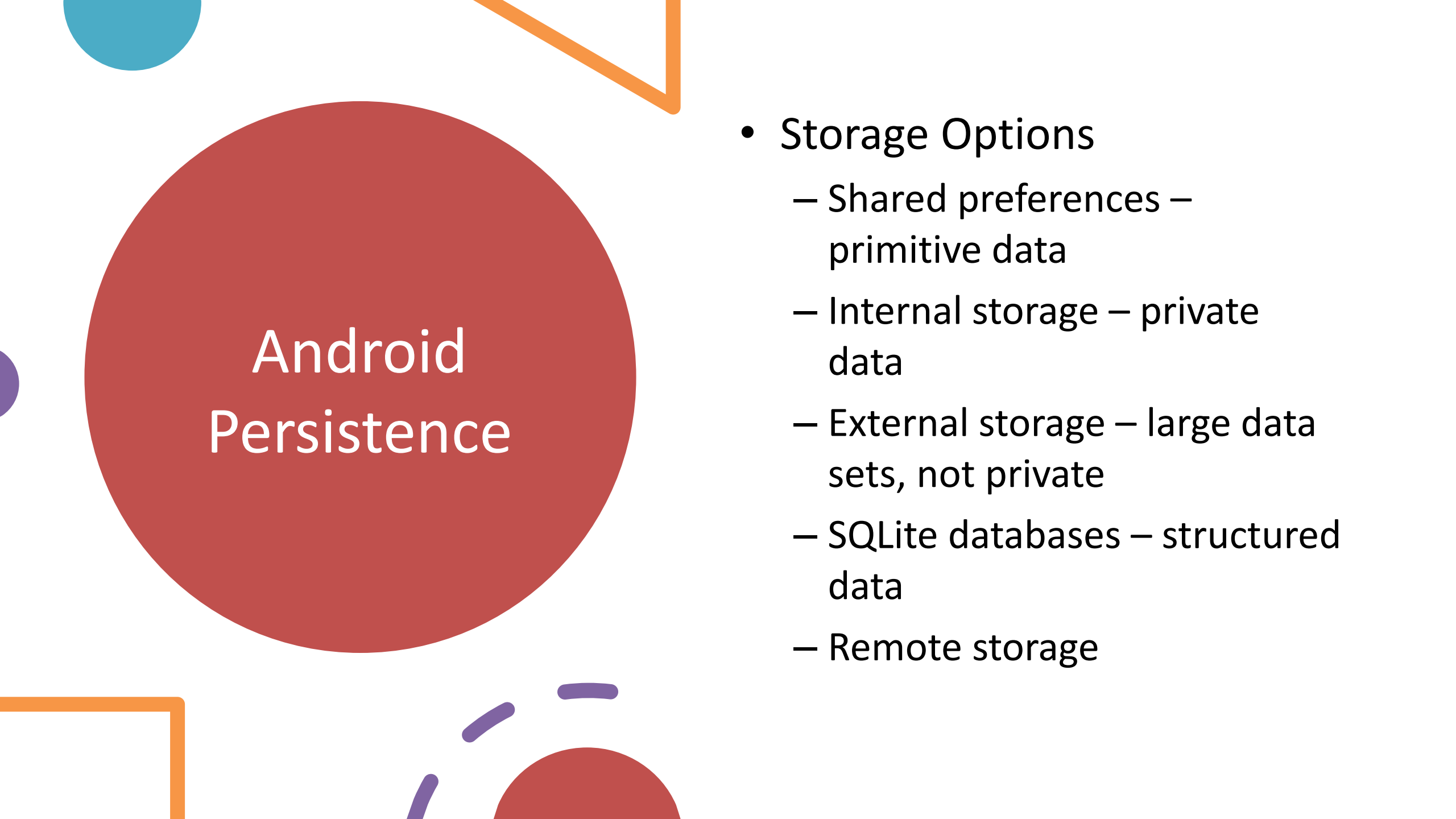
Overview of Persistency Options

SharedPreferences – key-value storage

Files & internal/external storage

SQLite – traditional relational DB

Room – modern ORM on top of SQLite



Android Persistence

- Storage Options
 - Shared preferences – primitive data
 - Internal storage – private data
 - External storage – large data sets, not private
 - SQLite databases – structured data
 - Remote storage

Shared preferences

- Primitive values: boolean, float, int, long and string
- Data persist across user sessions
- SharedPreferences objects
- `getSharedPreferences()`
- `getPreferences()`
- `edit()` – to get an editor
- put and get like methods
- commit to save data

Internal storage

- save files directly on the device's internal storage, private by default;
- The files are deleted after uninstalling the application
- `openFileOutput()` using a `FileOutputStream` object
- Write bytes into the file;
- `openFileInput()` using a `FileInputStream` object
- Read bytes from the file;
- Context modes: `PRIVATE`, `APPEND`, `WORLD_READABLE`, `WORLD_WRITEABLE`
- Also used for storing cache files with `getCacheDir()`

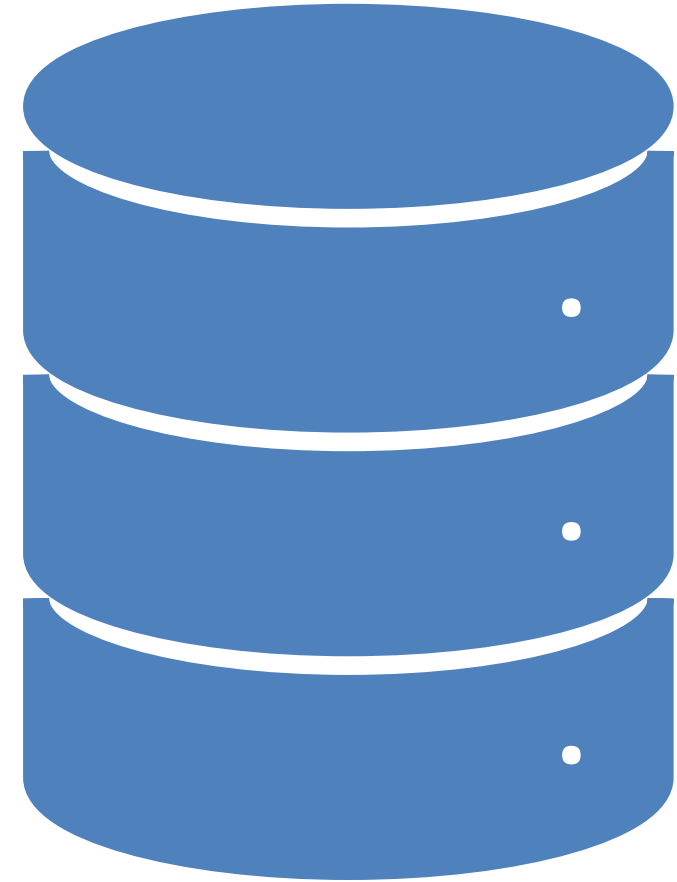
External storage

- An external storage represents a shared space such as an SD card, removable storage media, or an internal storage;
- Needs access to external storage declared in manifest file
 - `<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />`
- Check media availability
 - `getExternalStorageState()`
- Get the location for saving files:
 - `getExternalStoragePublicDirectory()`

- /data/data/package
- /data/media/0 - external
- /storage/emulated/0 - external
- /storage/emulated/0/Android/data/package – private external

SQLITE

STORING DATA IN DATABASES



Database

Android provides full support for SQLite

Relational databases

android.database.sqlite package

Uses SQLiteOpenHelper for creating a database

CRUD operations from SQLiteDatabase object

SQLite in Android

- Lightweight relational database
- Accessed via SQLiteOpenHelper
- Manual SQL queries
- Good for small–medium datasets

SQLite Definition

Self Contained

Minimal support from external libraries

Serverless

Reads and writes directly from the database files on disk

Zero-Configuration

No installation, no setup

Transactional

All changes in a transaction occur completely or not at all

SQLite

- Supported data types
 - INTEGER
 - REAL
 - TEXT
 - BLOB
 - NULL
- Restrictions:
 - NOT ALL JOINS are implemented;
 - Not all ALTER TABLE options are implemented, such as: DROP COLUMN, ALTER COLUMN, ADD CONSTRAINT
 - FOR EACH STATEMENT trigger is not supported;
 - VIEWS are READ ONLY OBJECTS
- Limitations:
 - max. length of a string or BLOB: $2^{31} - 1 = 2.147.483.647$ bytes
 - max. no. of columns / tables in a join / arguments in a function
 - max. no. of rows in a table: 2^{64}
 - max. database size: $2^{31} - 2$ pages / with 65536 bytes /page => 140 TB / 140.000 GB

SQLiteDatabase

- Create/Open Database
 - SQLiteDatabase **class**
 - **openDatabase()** static method
 - **openOrCreateDatabase()** static method
 - Context **class**
 - **openOrCreateDatabase()**
 - SQLiteOpenHelper **class**
 - **getReadableDatabase()**
 - **getWritableDatabase()**
- Close Database
 - **close()**

SQLite in Android App



Create a Contract Class

Create a class that Implements SQLiteOpenHelper

- onCreate(), onUpdate()
- Constructor

Get an SQLiteDatabase object

- insert(), update(), delete(), query()
- execSQL(), rawQuery()

Get an SQLiteCursor Object

Example

```
SQLiteDatabase db = SQLiteDatabase.openDatabase(  
    "pim.db", null);
```

```
//or
```

```
SQLiteDatabase db =  
    openOrCreateDatabase("pim.db",  
        Context.MODE_PRIVATE, null);
```

```
//...
```

```
db.close();
```

SQLiteDatabase

- SQL commands:
 - **execSQL()** – doesn't return any value
 - **rawQuery()** – return values (**Cursor**)
- Specialized methods
 - **query()**
 - **insert()**
 - **update()**
 - **delete()**

Database selection

- **query()** method
 - Table name
 - Selected columns
 - String[]
 - Selection criteria (WHERE)
 - String
 - Values associated with the each parameter from the selection criteria
 - String[]

Database selection

- Grouping (GROUP BY)
 - String
- Grouping condition (HAVING)
 - String []
- Sorting (ORDER BY)
 - String
- Return type:
 - Cursor

Selection example

```
Cursor cursor = db.query(  
    "students",  
    null,//all columns  
    null,//without selection  
    null,  
    null,//without grouping  
    null,  
    null//without sorting  
);
```

Selection example

```
Cursor cursor = db.query(  
    "students",  
    new String[] {"ids", "name", "faculty" }  
    "faculty=?",  
    new String[] {"CSIE"},  
    null,//without grouping  
    null,  
    "ids ASC"  
);
```

Database insertion

- **ContentValues** class
 - put("key", value);
- SQLiteDatabase#**insert()** method
 - table name
 - null – optional, for inserting null into an explicit column
 - **ContentValues** object
- Result:
 - Long – row ID of the newly inserted column or -1

Insertion example

```
ContentValues value = new ContentValues();
```

```
value.put("ids", 1200);
```

```
value.put("name", "Popescu");
```

```
long result = bd.insert("students" , null, value)
```

Database update

- **update()** method
 - table name
 - **ContentValues** object with the new values
 - WHERE clause, null value will update all rows
 - WHERE args, string values replacing the ? from the WHERE clause
 - returns an int representing the numbers of rows affected

Update example

```
ContentValues value = new ContentValues();
```

```
value.put("name", "new student");
```

```
int rez;
```

```
rez = db.update("students" , value, null, null);
```

```
rez = db.update("students" , value, "ids=?",  
                new String[] {String.valueOf(1200)});
```

Database deletion

- **delete()** method

- table name
- WHERE clause
- WHERE args

- Example

```
int rez;
```

```
rez = db.delete("students" , null, null);
```

```
rez = db.delete("students", "ids=?", new String[] {String.valueOf(1200)});
```

Transactions

- Transaction support:
 - beginTransaction()
 - setTransactionSuccessful()
 - applying changes made during transaction marking it as clean
 - endTransaction()

Transaction Example

```
SQLiteDatabase db = ...;
```

```
db.beginTransaction();
```

```
try {
```

```
    modifyDatabase();
```

```
    db.setTransactionSuccessful();
```

```
}catch (SQLException e) {
```

```
    //handling exceptions
```

```
}finally {
```

```
    db.endTransaction();
```

```
}
```

SQLiteOpenHelper

- A helper class to manage database creation and version management
- Create a subclass that implements the onCreate() and onUpgrade()
- Methods like
 - getReadableDatabase()
 - getWritableDatabase()
- used to actually create or open the database

Implementation

```
public class ManagerDB extends SQLiteOpenHelper {  
    int version = 1;  
    public ManagerDB (Context context) {  
        super(context, "pim.db", null, version);  
    }  
  
    @Override  
    public void onCreate(SQLiteDatabase db) {  
        try {  
            db.execSQL("CREATE TABLE faculties ...");  
        }  
        catch(SQLException ex) { ex.printStackTrace(); }  
    }  
}
```

Implementation

```
@Override
public void onUpgrade(SQLiteDatabase db, int prevVers, int newVers)
{
    try {
        db.execSQL("DROP TABLE IF EXISTS faculties");
        onCreate(db);
    }
    catch(SQLException ex) { ex.printStackTrace(); }
}
```

Example

```
ManagerDB mgrDB = new ManagerDB(this);
```

```
SQLiteDatabase db = mgrDB.getWritableDatabase();
```

```
...
```

```
mgrDB.close();
```

Cursor

- android.database
- Interface
- Manages the rows returned by a database selection
- SQLiteCursor
 - Cursor implementation for SQLite database

Cursor

- Crossing
 - **moveToNext()**
 - **moveToPrevious()**
 - **moveToFirst()**
 - **moveToLast()**
- Value extraction
 - **getInt(), getString(), getFloat(),** etc.
 - parameter: column index

Cursor

- Position testing:
 - **isFirst()**, **isLast()**
 - **isBeforeFirst()**, **isBeforeLast()**
- Extra info:
 - Number of records
 - **getCount()**
 - Column name
 - **getColumnName()**
 - Column position
 - **getColumnIndex()**

Cursor

- The **_id** column for the CursorAdapter
- SimpleCursorAdapter used to map columns to textview or imageview widgets defined in an XML
- Example

```
SimpleCursorAdapter adapter = new SimpleCursorAdapter  
    (this, // context  
     android.R.layout.two_line_list_item, //line layout  
     cursor, // the cursor  
     new String[] { ... } // columns name list,  
     new int[] { ... }  // identifiers list of associated resources  
     );
```

ANDROID EXTENSIONS – ROOM



Gradle Requirements

- **def** room_version = "2.2.2"

```
implementation "androidx.room:room-runtime:$room_version"  
annotationProcessor "androidx.room:room-compiler:$room_version"
```

Room Architecture Components



ROOM Components

```
@Database(entities =  
{User.class}, version = 1)  
public abstract class  
RepoDatabase extends RoomDatabase
```

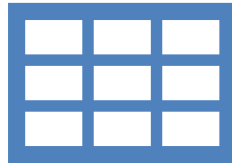
```
@Entity(tableName = "users")  
public class User
```

```
@Dao  
public interface UserDao
```

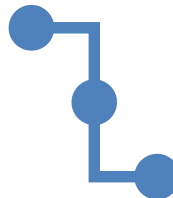
ROOM



@Database
class implementation



@Entity
class implementation



@Dao
Interface

Room Persistence Library

- Abstraction over SQLite
- Compile-time SQL validation
- Entities, DAO interfaces, Database class
- Supports LiveData / Flow

Best Practices

- Use Room instead of raw SQLite
- Avoid running DB on main thread
- Use coroutines or RxJava
- Perform migrations carefully

Topics

- Type converters
- Database relationships
- Prepopulating databases
- Optimizing DAOs using @Transaction and batching
- Handling complex relations (1–N, N–N)
- Schema versioning with auto-migrations and manual migrations
- In-memory & encrypted Room DBs (SQLCipher)

Type Converters

- Room only stores primitive types + String.
- TypeConverters allow storing custom/complex types.
- Used to convert between Kotlin/Java objects and DB-supported formats.
- Applied with `@TypeConverter` or `@TypeConverters`.

DateConverter

```
public class DateConverter {  
  
    @TypeConverter  
    public static Date fromTimestamp(Long value) {  
        return value == null ? null : new Date(value);  
    }  
  
    @TypeConverter  
    public static Long dateToTimestamp(Date date) {  
        return date == null ? null : date.getTime();  
    }  
}
```

ListConverter

```
public class ListConverter {  
  
    private static final Gson gson = new Gson();  
  
    @TypeConverter  
    public static List<String> fromJson(String value) {  
        Type listType = new TypeToken<List<String>>().getType();  
        return gson.fromJson(value, listType);  
    }  
  
    @TypeConverter  
    public static String listToJson(List<String> list) {  
        return gson.toJson(list);  
    }  
}
```

Types of Database Relationships

- One-to-One: Entity A $\leftrightarrow$ Entity B
- One-to-Many: Parent $\leftrightarrow$ List<Child>
- Many-to-Many: Requires join table
- Handled via @Relation annotations (Room)

Prepopulating Databases

- Useful for initial datasets, lookup tables, default values
- Can be done using `RoomDatabase.Callback`
- Prepopulation runs only on first creation

Room Database Callback

```
Room.databaseBuilder(context, DatabaseManager.class, name: "app.db")
    .addCallback(new RoomDatabase.Callback() {
        @Override 1 usage
        public void onCreate(@NonNull SupportSQLiteDatabase db) {
            super.onCreate(db);
            Executors.newSingleThreadExecutor().execute(() -> {
                db.execSQL(s: "INSERT INTO users (id, name) VALUES (1, 'Admin')");
            });
        }
    })
    .build();
```

Transactions

Authors

Books

```
public class AuthorWithBooks {  
    @Embedded  
    public Author author;  
  
    @Relation(  
        parentColumn = "authorId",  
        entityColumn = "authorId"  
    )  
    public List<Book> books;  
}
```

@Entity Definition

```
@Entity(tableName = "users")
@Entity(primaryKeys = {"firstName", "lastName"})
@Entity(indices = {@Index(value = {"first_name",
    "last_name"},
    unique = true))})
```

Attributes / Methods annotations:

- @NotNull
- @PrimaryKey(autoGenerate = true)
- @ColumnInfo(name = "userid")
- @Ignore

@Dao Implementation

```
@Query("SELECT * FROM Users WHERE username=:name")
```

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
@Update
```

- OnConflictStrategy.REPLACE
- OnConflictStrategy.ROLLBACK
- OnConflictStrategy.ABORT
- OnConflictStrategy.FAIL
- OnConflictStrategy.IGNORE

```
@Delete
```

```
@Query("DELETE FROM Users")
```

@Query

As an extension over SQLite bind arguments, Room supports binding a list of parameters to the query. At runtime, Room will build the correct query to have matching number of bind arguments depending on the number of items in the method parameter.

```
@Query("SELECT * FROM user WHERE uid IN(:userIds)")  
public abstract List findByIds(int[] userIds);
```

@Insert

Marks a method in a `Dao` annotated class as an insert method.

The implementation of the method will insert its parameters into the database.

All of the parameters of the Insert method must either be classes annotated with `Entity` or collections/array of it.

Example:

```
@Dao
public interface MyDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    public void insertUsers(User... users);
    @Insert
    public void insertBoth(User user1, User user2);
    @Insert
    public void insertWithFriends(User user, List<User> friends);
}
```

@Delete

Marks a method in a `Dao` annotated class as a delete method.

The implementation of the method will delete its parameters from the database.

All of the parameters of the Delete method must either be classes annotated with `Entity` or collections/array of it.

Example:

```
@Dao
public interface MyDao {
    @Delete
    public void deleteUsers(User... users);
    @Delete
    public void deleteAll(User user1, User user2);
    @Delete
    public void deleteWithFriends(User user, List<User> friends);
}
```

Resources

- C. Boja, C. Ciurea, M. Doinea – Android Mobile Applications: A Practical Development Guide, ASE Press, 2015, 404 pp., ISBN 978-606-34-0033-9
- S. Komatineni, D. MacLean – Pro Android 4, Apress, 2012
- R. Meier – Professional Android 4 Application Development, Wiley, 2012
- P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012
- P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015
- <http://developer.android.com>